

# La programmation orientée objet - Encapsulation

---

L'encapsulation est un des 3 grands principes de la programmation orientée objet (POO), avec l'héritage et le polymorphisme.

Cela consiste à protéger les données d'un objet pour éviter qu'elles ne soient modifiées directement depuis l'extérieur de l'objet.

Dans plusieurs langages de programmation orientée objet, comme Java ou C#, il est possible de définir des attributs privés qui ne peuvent être accédés directement depuis l'extérieur de l'objet.

En JavaScript, c'est plus complexe.

## Propriétés publiques et privées

Une propriété publique est une propriété qui peut être accédée et modifiée directement depuis l'extérieur de l'objet.

Ex:

```
let personne = new Personne("Jean", 30);
personne.nom = "Paul"; // Propriété publique
personne.age = 25; // Propriété publique
personne.direBonjour(); //Méthode publique
```

Une propriété privée est une propriété qui ne peut être accédée et modifiée que depuis l'intérieur de l'objet.

Il existe depuis peu, une nouvelle fonctionnalité en JavaScript qui permet de déclarer des propriétés privées et des méthodes dans une classe.

On préfixe simplement le nom de la propriété par un `#` pour la rendre privée. Elle sera visible dans la console, mais impossible à modifier. À noter, qu'il faut aussi les déclarer avant le constructeur.

Ex:

```
class Personne {
  #nom;
  #age;

  constructor(nom, age) {
    this.#nom = nom;
    this.#age = age;
  }

  #formaterNom(nom) {
    return nom.toUpperCase();
  }

  direBonjour() {
```

```
        console.log(`Bonjour, je m'appelle ${this.#nom} et j'ai ${this.#age}
ans.`);
    }
}
```

## Pourquoi l'encapsulation ?

L'encapsulation permet de protéger les données de l'objet en les rendant privées. Cela permet de contrôler l'accès aux données et de garantir leur intégrité.

Il faut prendre pour acquis que les développeurs qui utiliseront votre code pourrait faire n'importe quoi avec les données de l'objet. L'encapsulation permet de limiter les risques d'erreurs et de bugs en contrôlant l'accès aux données.

Une bonne pratique est toujours de déclarer les propriétés d'un objet comme privées et de fournir des méthodes publiques pour accéder et modifier ces propriétés. Cela permet de garantir que les données de l'objet sont toujours valides et cohérentes.

### Accesseurs et mutateurs (Getters et Setters)

Les accesseurs et les mutateurs sont des méthodes spéciales qui permettent d'accéder et de modifier les propriétés privées d'un objet.

Les accesseurs sont des méthodes qui permettent de lire la valeur d'une propriété privée. Ils sont généralement appelés des **getters**.

Les mutateurs sont des méthodes qui permettent de modifier la valeur d'une propriété privée. Ils sont généralement appelés des **setters**. Ils permettent également de valider les données avant de les modifier et les formater si nécessaire.

En JavaScript, on utilise les mots-clés **get** et **set** pour définir des accesseurs et des mutateurs.

Ex:

```
class Personne {
    #nom; //Propriété privée
    #age; //Propriété privée

    constructor(nom, age) {
        this.#nom = nom;
        this.#age = age;
    }

    // La propriété nom est en lecture seule (aucun setter), il n'est plus
    // possible de la modifier après la création de l'objet
    get nom() {
        return this.#nom;
    }

    get age() {
        return this.#age;
    }
}
```

```

    }

    set age(age) {
        // Validation simple de l'âge
        if (age < 0) {
            throw new Error("L'âge ne peut pas être négatif.");
            return;
        }
        this.#age = age;
    }

    // Cette méthode est privée et ne peut pas être appelée depuis l'extérieur de
    la classe
    #formaterNom(nom) {
        return nom.toUpperCase();
    }

    // Cette méthode est publique et peut être appelée depuis l'extérieur de la
    classe
    direBonjour() {
        console.log(`Bonjour, je m'appelle ${this.#nom} et j'ai ${this.#age}
ans.`);
    }
}

```

Pourquoi ne pas utiliser des propriétés publiques alors? Cela permet de limiter les risques d'erreurs et de bugs en contrôlant l'accès aux données.

Ex: on pourrait créer une propriété en lecture seule en ne définissant que le getter et aucun setter.

On pourrait aussi formater la valeur avant de la retourner ou de la modifier dans le setter.

## Utiliser les mutateurs et accesseurs

Pour accéder à une propriété privée, on utilise les accesseurs (getters) et pour modifier une propriété privée, on utilise les mutateurs (setters) sans les parenthèses comme s'il s'agissait d'une propriété publique.

```

let personne = new Personne("Jean", 30);
console.log(personne.nom); // Jean
console.log(personne.age); // 30

personne.nom = "Paul";
personne.age = 25;

console.log(personne.nom); // Paul
console.log(personne.age); // 25
personne.direBonjour(); // Bonjour, je m'appelle Paul et j'ai 25 ans.

```

## Exemple d'un module utilisant l'encapsulation

```

class Compteur {
    #temps = 0;
    #intervalle = null;
    #conteneurHTML = null;
    #elementHTML = null;

    constructor(conteneurHTML) {
        this.#conteneurHTML = conteneurHTML;
        this.#temps = 0;
        this.#intervalle = null;
        this.#elementHTML = null;

        this.#genererHTML();
    }

    /**
     * Retourne le temps formaté en mm:ss
     */
    get temps() {
        const minutes = Math.floor(this.#temps / 60);
        const secondes = this.#temps % 60;

        const tempsFormatte = `${minutes.toString().padStart(2, "0")}:${secondes.toString().padStart(2, "0")}`;

        return tempsFormatte;
    }

    /**
     * Génère le HTML du compteur et attache les événements
     */
    #genererHTML() {
        const gabarit = `<div data-compteur>Temps écoulé : <span data-conteneur-temps>${this.temps}</span></div>`;

        this.#conteneurHTML.insertAdjacentHTML("beforeend", gabarit);
        this.#elementHTML = this.#conteneurHTML.lastElementChild;

        this.#elementHTML.addEventListener("click", this.demarrer.bind(this));
    }

    /**
     * Met à jour le HTML du compteur
     */
    #mettreAJourHTML() {
        this.#elementHTML.querySelector("[data-conteneur-temps]").textContent = this.temps;
    }

    /**
     * Démarre le compteur
     */
    demarrer() {
        this.arreter();
    }

```

```
        this.#intervalle = setInterval(() => {
            this.#temps++;
            this.#mettreAJourHTML();
        }, 1000);
    }

    /**
     * Arrête le compteur
     */
    arreter() {
        clearInterval(this.#intervalle);
        this.#intervalle = null;
        this.#temps = 0;
    }
}
```